

SAT and CNF Exercises

Question–Answer Notes for Exercises 34.4-1 to 34.4-7

Background

Main Theme

These exercises explain why the careful reductions to SAT and 3-CNF-SAT are needed, and they contrast hard satisfiability variants with easy ones such as DNF satisfiability and 2-CNF satisfiability.

1 Exercise 34.4-1

Question 34.4-1

Consider the straightforward nonpolynomial-time reduction in the proof of Theorem 34.9. Describe a circuit of size n that, when converted to a formula by this method, yields a formula whose size is exponential in n .

Answer

Use a circuit with repeated fan-out. Let the first gate compute

$$g_0 = x_1 \wedge x_2.$$

For each $i = 1, 2, \dots, n$, create one new AND gate

$$g_i = g_{i-1} \wedge g_{i-1},$$

using the output wire of g_{i-1} twice as input to g_i .

Why the Formula Blows Up

The circuit has only $\Theta(n)$ gates. But the direct recursive conversion to a formula gives

$$F_i = F_{i-1} \wedge F_{i-1}.$$

If s_i is the size of F_i , then

$$s_i = 2s_{i-1} + 1.$$

Hence

$$s_i = \Theta(2^i).$$

So a circuit of size $\Theta(n)$ can become a formula of size $\Theta(2^n)$ under the straightforward expansion.

2 Exercise 34.4-2

Question 34.4-2

Show the 3-CNF formula that results when we use the method of Theorem 34.10 on the formula

$$\phi = ((x_1 \Rightarrow x_2) \vee \neg((\neg x_1 \Leftrightarrow x_3) \vee x_4)) \wedge \neg x_2.$$

Answer

The parse-tree construction gives

$$\begin{aligned} \phi' = & y_1 \wedge (y_1 \Leftrightarrow (y_2 \wedge \neg x_2)) \wedge (y_2 \Leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \Leftrightarrow (x_1 \Rightarrow x_2)) \wedge (y_4 \Leftrightarrow \neg y_5) \\ & \wedge (y_5 \Leftrightarrow (y_6 \vee x_4)) \wedge (y_6 \Leftrightarrow (\neg x_1 \Leftrightarrow x_3)). \end{aligned}$$

Using the theorem's conversion, the final 3-CNF formula can be written as

$$\psi = P_1 \wedge E_1 \wedge E_2 \wedge E_3 \wedge E_4 \wedge E_5 \wedge E_6,$$

where p and q are fresh variables and the groups below are all 3-literal clauses.

Root Clause

$$\begin{aligned} P_1 = & (y_1 \vee p \vee q) \wedge (y_1 \vee p \vee \neg q) \\ & \wedge (y_1 \vee \neg p \vee q) \wedge (y_1 \vee \neg p \vee \neg q). \end{aligned}$$

Gate Clauses

$$\begin{aligned} E_1 = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2), \\ E_2 = & (\neg y_2 \vee y_3 \vee y_4) \wedge (y_2 \vee \neg y_3 \vee \neg y_4) \\ & \wedge (y_2 \vee \neg y_3 \vee y_4) \wedge (y_2 \vee y_3 \vee \neg y_4), \\ E_3 = & (y_3 \vee x_1 \vee x_2) \wedge (y_3 \vee x_1 \vee \neg x_2) \\ & \wedge (y_3 \vee \neg x_1 \vee \neg x_2) \wedge (\neg y_3 \vee \neg x_1 \vee x_2), \\ E_4 = & (y_4 \vee y_5 \vee p) \wedge (y_4 \vee y_5 \vee \neg p) \\ & \wedge (\neg y_4 \vee \neg y_5 \vee p) \wedge (\neg y_4 \vee \neg y_5 \vee \neg p), \\ E_5 = & (\neg y_5 \vee y_6 \vee x_4) \wedge (y_5 \vee \neg y_6 \vee \neg x_4) \\ & \wedge (y_5 \vee \neg y_6 \vee x_4) \wedge (y_5 \vee y_6 \vee \neg x_4), \\ E_6 = & (\neg y_6 \vee \neg x_1 \vee \neg x_3) \wedge (\neg y_6 \vee x_1 \vee x_3) \\ & \wedge (y_6 \vee \neg x_1 \vee x_3) \wedge (y_6 \vee x_1 \vee \neg x_3). \end{aligned}$$

Reading the Formula

Each E_i is the 3-CNF encoding of one local equivalence in ϕ' . The group P_1 is the padded version of the root requirement y_1 .

3 Exercise 34.4-3

Question 34.4-3

Professor Jagger proposes to show $\text{SAT} \leq_p \text{3-CNF-SAT}$ using only the truth-table technique from Theorem 34.10. Show that this strategy does not yield a polynomial-time reduction.

Answer

The strategy fails because the truth table of the whole formula can be exponentially large.

Reason

An input formula with n variables has

$$2^n$$

truth assignments. The proposed reduction builds a truth table over all variables, then constructs a DNF formula for the falsifying rows and negates it to obtain a CNF formula.

Even if the original formula has length polynomial in n , explicitly listing the truth table already takes $\Omega(2^n)$ time and space. Therefore the resulting formula can have exponentially many clauses, and the construction is not a polynomial-time reduction.

4 Exercise 34.4-4

Question 34.4-4

Show that the problem of determining whether a Boolean formula is a tautology is complete for **co-NP**.

Answer

Let

$$\text{TAUTOLOGY} = \{\langle \phi \rangle \mid \phi \text{ is true under every assignment}\}.$$

We prove that TAUTOLOGY is in **co-NP** and is **co-NP**-hard.

TAUTOLOGY Is in co-NP

The complement of TAUTOLOGY is the set of formulas that are false under at least one assignment. A certificate is such an assignment, and the verifier evaluates the formula and checks that it gives 0. Thus

$$\overline{\text{TAUTOLOGY}} \in \text{NP},$$

so

$$\text{TAUTOLOGY} \in \text{co-NP}.$$

TAUTOLOGY Is co-NP-Hard

Since SAT is **NP**-complete, its complement UNSAT is **co-NP**-complete. Reduce UNSAT to TAUTOLOGY by mapping

$$\phi \mapsto \neg\phi.$$

Then

$$\phi \in \text{UNSAT} \iff \phi \text{ is unsatisfiable} \iff \neg\phi \text{ is a tautology}.$$

The map $\phi \mapsto \neg\phi$ is computable in polynomial time. Hence

$$\text{UNSAT} \leq_p \text{TAUTOLOGY}.$$

Therefore TAUTOLOGY is **co-NP**-hard. Since it is also in **co-NP**, it is **coNP**-complete.

5 Exercise 34.4-5

Question 34.4-5

Show that determining satisfiability of Boolean formulas in disjunctive normal form is polynomial-time solvable.

Answer

A DNF formula has the form

$$T_1 \vee T_2 \vee \cdots \vee T_m,$$

where each term T_i is an AND of literals.

Algorithm

A DNF formula is satisfiable iff at least one term T_i is satisfiable.

For each term T_i , check whether it contains both a variable and its negation.

- If T_i contains both x and $\neg x$, then T_i is unsatisfiable.
- If no such conflict occurs, then T_i is satisfiable by assigning each literal in the term to be true.

Accept iff some term has no conflict. This scan takes polynomial time, and with simple marker arrays it can be implemented in linear time in the formula length.

6 Exercise 34.4-6

Question 34.4-6

Suppose someone gives you a polynomial-time algorithm to decide formula satisfiability. Describe how to use it to find satisfying assignments in polynomial time.

Answer

Use the SAT decider repeatedly to fix variables one at a time.

Self-Reduction Algorithm

Let A be a polynomial-time SAT decider. Given a formula ϕ :

1. Run $A(\phi)$. If ϕ is not satisfiable, report that no satisfying assignment exists.
2. Let the variables be $x_1, x_2, \dots, x_n$.
3. For $i = 1, 2, \dots, n$:
 - a. Temporarily set $x_i = 1$ in the current formula.
 - b. Run A on the restricted formula.

- c. If it is satisfiable, permanently set $x_i = 1$.
- d. Otherwise, permanently set $x_i = 0$.

4. Return the resulting assignment.

At each step, the current partial assignment is extendable to a satisfying assignment. If setting $x_i = 1$ preserves satisfiability, keep it. Otherwise, setting $x_i = 0$ must preserve satisfiability. The algorithm makes at most $n+1$ calls to the polynomial-time decider and does only polynomial additional work. Hence it runs in polynomial time.

7 Exercise 34.4-7

Question 34.4-7

Let 2-CNF-SAT be the set of satisfiable Boolean formulas in CNF with exactly two literals per clause. Show that

$$2\text{-CNF-SAT} \in \mathbf{P}.$$

Make the algorithm as efficient as possible.

Answer

Reduce 2-CNF-SAT to strongly connected components in an implication graph.

Implication Graph

For each variable x_i , create two vertices:

$$x_i \quad \text{and} \quad \neg x_i.$$

For each clause

$$(a \vee b),$$

add the two directed edges

$$\neg a \rightarrow b \quad \text{and} \quad \neg b \rightarrow a.$$

This is valid because

$$a \vee b \quad \equiv \quad (\neg a \Rightarrow b) \wedge (\neg b \Rightarrow a).$$

Decision Rule

Compute the strongly connected components of the implication graph.

The formula is satisfiable iff no variable x_i has both x_i and $\neg x_i$ in the same strongly connected component.

Correctness

If x_i and $\neg x_i$ are in the same SCC, then each implies the other. Any satisfying assignment would force both x_i and $\neg x_i$ to have the same truth value, which is impossible.

Conversely, if no variable is in the same SCC as its negation, then a satisfying assignment can be obtained from the SCC condensation DAG. Topologically order the SCCs, and set a literal to true if its SCC appears after the SCC of its negation. This assignment respects every implication edge, and therefore satisfies every clause.

Running Time

The graph has $2n$ vertices and $2m$ edges for n variables and m clauses. Strongly connected components can be found in

$$\mathcal{O}(n + m)$$

time using Tarjan's or Kosaraju's algorithm. Thus

$$2\text{-CNF-SAT} \in \mathbf{P},$$

and in fact it is solvable in linear time.

Final Summary**What to Remember**

- Naively expanding circuits into formulas can duplicate shared subcircuits exponentially many times.
- Theorem 34.10 avoids exponential blowup by using local constraints and then converting only constant-size pieces to CNF.
- A whole-formula truth table is too large for a polynomial reduction.
- TAUTOLOGY is **co-NP**-complete.
- DNF satisfiability is easy: check for one consistent term.
- A SAT decider can be turned into a satisfying-assignment finder by fixing variables one at a time.
- 2-CNF-SAT is linear-time solvable using SCCs in the implication graph.